CS683: Advanced Computer Architecture

Programming Assignment 1

TASK1 - 2D Convolution Optimization

Group: Runtime Terrors

Group Members

- Druthi Sree Somineni – 24b1252
- Urjith Karthikeya – 24b1308
- Harshvardhan Patil – 24b1205
- Deepika Kale – 24b1203

Task 1A: Loop Unrolling and Loop Reordering

What motivated you to make changes to your code?

In the naive implementation, for every output element we traverse the entire kernel along with its corresponding input region. This means that the same kernel elements are accessed repeatedly for different output elements, while the input accesses are spread across different regions.
The loop performs a comparison of i(current value) with the matrix size for every iteration, increasing the overhead for each iteration and resulting in increased execution time.

What considerations did you take into account when implementing those changes?

By loop reordering, we changed the loop order to process one kernel element at a time and apply it across all the output elements. This makes the input and output accesses more sequential along each row, providing better spatial locality, while the current kernel value can be reused across the sweep.

By loop unrolling, rather than comparing i with n every time, we took advantage of the fact that the size is always in multiples of 8 and also the spatial locality of the array, which decreases the number of comparison instructions it should execute, leading to a decrease in execution time

How effective are your changes? Which metric will you use to quantify effectiveness and why? We use execution time and speedup relative to the naive implementation as the primary metrics to quantify the effectiveness of our changes. Execution time directly measures the time taken to perform the convolution, while speedup shows the relative improvement when compared to the naive implementation

Results

Table 1: Performance comparison for 256×256 matrix

Method	Run 1		Run 2		Run 3	
	Time (ms)	Speedup	Time (ms)	Speedup	Time (ms)	Speedup
Naive	0.613	1.00×	0.282	1.00×	0.769	1.00×
Unroll	0.202	3.03×	0.181	1.56×	0.231	3.33×
Reorder	0.482	1.27×	0.186	1.51×	0.481	1.60×

Table 2: Performance comparison for 512×512 matrix

Method	Run 1		Run 2		Run 3	
	Time (ms)	Speedup	Time (ms)	Speedup	Time (ms)	Speedup
Naive	1.922	1.00×	1.213	1.00×	1.190	1.00×
Unroll	0.336	3.26×	0.337	3.60×	0.336	3.54×
Reorder	0.701	2.74×	0.695	1.74×	0.691	1.72×

Table 3: Performance comparison for 1024×1024 matrix

Method	Run 1		Run 2		Run 3	
	Time (ms)	Speedup	Time (ms)	Speedup	Time (ms)	Speedup
Naive	4.447	1.00×	4.454	1.00×	4.475	1.00×
Unroll	1.333	3.34×	1.330	3.35×	1.330	3.36×
Reorder	2.870	1.55×	2.754	1.62×	3.546	1.26×

Task 1B: Tiling

The L1 Data cache size of the system was determined to be 48KIB per core

$$\boxed{L1D = [48\text{KiB/core}]}$$

Baseline L1-D MPKI

The baseline (naive) implementation was profiled with `perf`, averaging over five runs

The MPKI is computed as

$$\text{MPKI} = \frac{\text{L1-D load misses}}{\text{Instructions}} \times 1000.$$

The number of instructions, L1-D misses, MPKI, and the time for execution for naive execution are given here for the baseline

Matrix Size	**Instructions**	**L1-D Misses**	**L1-D MPKI**	**Time (ms)**
256×256	74,507,173	59,404	0.797	1.098
512×512	272,950,714	216,156	0.792	4.690
1024×1024	1,066,549,216	527,788	0.495	11.529
2048×2048	4,199,837,542	975,905	0.232	34.429
4096×4096	16,611,890,019	6,017,252	0.362	129.163

Tiled Implementation

Implemented the tiled version of the 2D convolution. For each tile size, implemented it for different matrix sizes and created the plots comparing L1-D MPKI vs. matrix size for each tile size, and similarly for the speedup.

L1-D MPKI for Different Tile Sizes

Table 4: L1-D MPKI for different tile sizes, with the naive baseline for reference.

Matrix Size	8×8	16×16	32×32	64×64	128×128	Naive
256×256	0.590	0.794	0.670	0.588	0.720	0.797
512×512	0.859	0.501	0.911	0.734	0.646	0.792
1024×1024	0.629	1.310	0.887	0.754	0.908	0.495
2048×2048	0.331	1.033	0.749	0.379	0.434	0.232
4096×4096	**0.256**	1.015	0.571	0.371	0.315	0.362

L1-D MPKI vs. Matrix Size

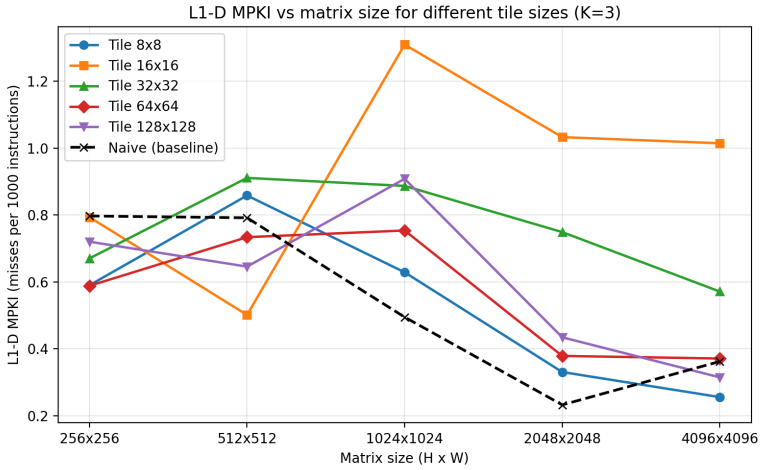

Figure 1: L1-D cache MPKI versus matrix size for different tile sizes. The dashed black curve is the naive baseline.

Execution Time for Different Tile Sizes

Table 5: Execution time (ms) of the tiled convolution for different tile sizes, with the naive reference measured in the same process.

Matrix Size	Naive	8×8	16×16	32×32	64×64	128×128
256×256	0.891	1.121	0.682	0.492	0.621	0.578
512×512	2.959	4.097	2.248	2.180	1.983	1.896
1024×1024	7.961	10.820	8.990	8.168	7.823	7.223
2048×2048	32.250	43.588	36.377	33.008	31.810	29.565
4096×4096	127.979	174.090	145.188	133.519	129.736	117.845

Table 6: Speedup of the tiled implementation over the naive baseline, $S = T_{\text{naive}}/T_{\text{tiled}}$.

Matrix Size	8×8	16×16	32×32	64×64	128×128
256×256	0.795	1.025	0.986	1.055	1.175
512×512	0.722	0.879	1.164	1.355	1.506
1024×1024	0.736	0.875	0.973	1.020	**1.075**
2048×2048	0.740	0.879	0.973	0.997	**1.070**
4096×4096	0.735	0.883	0.955	0.995	**1.074**

Speedup vs. Matrix Size

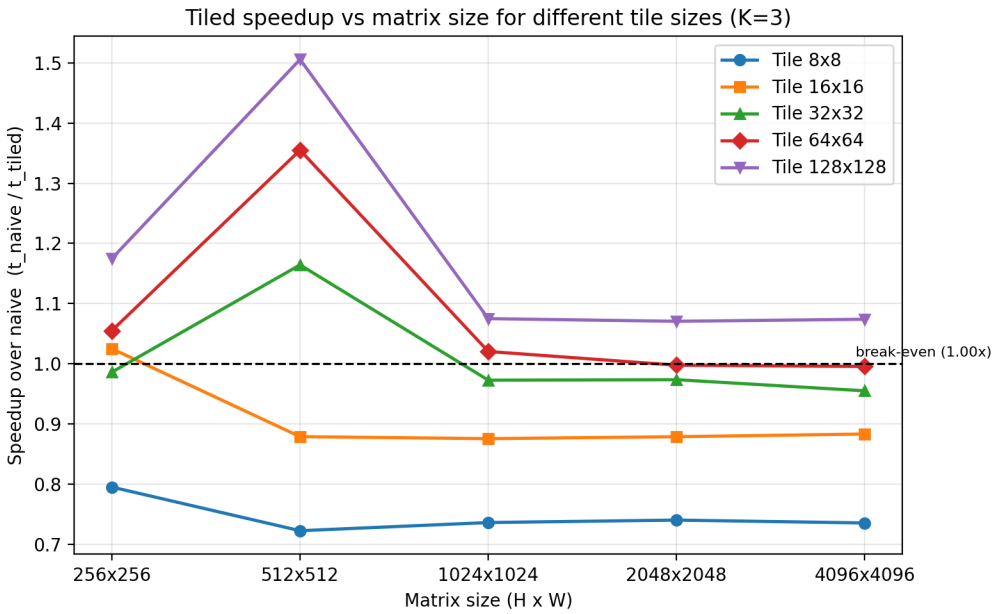

Figure 2: Speedup of tiled 2D convolution relative to the naive implementation for different tile sizes. The dashed line marks break-even.

Best Tile Size

Taking execution time as the primary criterion and MPKI as the secondary one:

$$\boxed{B_{\text{best}} = 128 \times 128}$$

At 4096×4096:

$$T_{\text{baseline}} = 127.979 \text{ ms}, \qquad T_{\text{best}} = 117.845 \text{ ms}$$

$$S_{\text{best}} = \frac{T_{\text{baseline}}}{T_{\text{best}}} = \frac{127.979}{117.845} = 1.074$$

Question 1: Naive vs. Tiled L1-D MPKI

Report the changes in L1-D MPKI observed when moving from naive to tiled 2D convolution. Justify your observations.

The L1-D MPKI did not consistently decrease after applying tiling. The result depended on both the matrix size and the chosen tile size.

For example, for the 256×256 matrix, the naive implementation had an MPKI of 0.797. Most tiled versions performed better, with the 64×64 tile reducing the MPKI to 0.588. Similarly, for 512×512, the 16×16 tile reduced the MPKI from 0.792 to 0.501. However, this improvement was not seen for every configuration. At 2048×2048, the naive implementation had an MPKI of 0.232, while all the tiled versions had a higher MPKI. At 4096×4096, the 8×8 tile reduced the MPKI from 0.362 to 0.256, whereas the 16×16 tile increased it to 1.015.

Thus, tiling by itself does not guarantee a reduction in L1-D misses.

The main reason is that the naive implementation of the 3×3 convolution already has good cache locality. Since the matrices are traversed in row-major order, consecutive output elements access mostly consecutive input elements, giving good spatial locality. Also, neighbouring output elements reuse many of the same input values, giving temporal locality. Therefore, the naive implementation is already able to make good use of the cache.

Tiling can improve locality by restricting the computation to a smaller region of the matrix at a time. However, it also introduces some additional effects. Neighbouring tiles have overlapping input regions because of the convolution boundary, so some input elements can be accessed again when processing the next tile. Smaller tiles also create more tile boundaries and shorter sequential access regions.

The MPKI values are also not monotonic with tile size. For example, the 16×16 tile gives a much larger MPKI than both smaller and larger tiles for several matrix sizes. This suggests that factors other than just the tile size, such as cache accesses, hardware prefetches, and tile traversal patterns with respect to cache size may also affect L1-D misses.

Overall, it shows that the naive convolution already has good locality, so tiling provides an improvement only for some configurations. Depending on the tile size, the benefit of working on a smaller region can be offset by repeated accesses and changes in the memory access pattern.

Question 2: Effect of Matrix Size and Tile Size

How did L1-D MPKI vary across different matrix sizes and tile sizes? Explain your findings in terms of cache hierarchy and working set sizes.

The L1-D MPKI varies with both the matrix size and the tile size, but the variation is not monotonic. This indicates that the cache behaviour is affected not only by the amount of data being processed, but also by the particular memory access pattern generated by each tile size.

First, considering the naive implementation, the measured MPKI values were 0.797, 0.792, 0.495, 0.232, and 0.362 for matrix sizes from 256×256 to 4096×4096. Thus, the MPKI decreases as the matrix size increases up to 2048×2048, but increases again for the 4096×4096 matrix.

For a 3×3 convolution, the complete matrix does not need to fit in L1 cache. The more relevant working set is the small number of input rows that are reused while producing consecutive output rows. Approximately three input rows are actively reused. For a matrix width W, these rows occupy

$$3 \times W \times 4 \text{ bytes},$$

assuming single-precision floating-point elements.

For example,

$$\begin{aligned} W = 1024: &\quad 3 \times 1024 \times 4 = 12 \text{ KiB}, \\ W = 2048: &\quad 3 \times 2048 \times 4 = 24 \text{ KiB}, \\ W = 4096: &\quad 3 \times 4096 \times 4 = 48 \text{ KiB}. \end{aligned}$$

The measured L1-D cache size is 48 KiB. Therefore, for widths up to 2048, the input rows that are being reused can fit comfortably in L1 cache. At 4096×4096, however, the three input rows alone occupy approximately the entire L1-D cache. Additional data such as output values and other memory accesses can then create cache pressure. This is consistent with the rise in the naive MPKI from 0.232 at 2048×2048 to 0.362 at 4096×4096.

The effect of tile size is more complicated. With tiling, only a small portion of the matrix is processed at a time. For a $B \times B$ output tile, a 3×3 convolution requires approximately a $(B+2) \times (B+2)$ region of the input matrix. The size of this input region is therefore

$$(B+2)^2 \times 4 \text{ bytes}.$$

For the tile sizes used in the experiment, the approximate input working-set sizes are

Tile Size	Input Region Size
8×8	0.39 KiB
16×16	1.27 KiB
32×32	4.52 KiB
64×64	17.0 KiB
128×128	66.0 KiB

Thus, the input working sets of the 8, 16, 32, and 64 sized tiles are small enough to fit within the 48 KiB L1-D cache. The 128×128 input region is larger than L1 and therefore cannot reside completely in L1 at once. However, this does not mean that the 128×128 tile must perform badly, since the computation still accesses the tile sequentially and data that is not present in L1 can be supplied by the lower levels of the cache hierarchy.

The measured results also show that smaller working sets do not always produce lower MPKI. For example, at 4096×4096, the MPKI values were

$$0.256,\ 1.015,\ 0.571,\ 0.371,\ 0.315$$

for tile sizes 8, 16, 32, 64, and 128, respectively. The 16×16 tile has the highest MPKI even though its working set is very small. Therefore, capacity alone cannot explain the observed behaviour. Other effects such as cache-set mapping, the exact row stride, hardware prefetching, and repeated accesses near tile boundaries can also affect the number of L1-D misses.

Overall, increasing the matrix size increases cache pressure once the reused row working set approaches the L1-D capacity. Tiling reduces the active working set and can therefore help at large matrix sizes, but the MPKI is not determined by working-set size alone. The access pattern generated by the tile size and its interaction with the cache hierarchy are also important, which is why the measured MPKI is not monotonic with tile size.

Question 3: Speedup

Did you achieve a speedup? If yes, quantify the improvement and identify the contributing factors. If not, analyze the limiting factors.

Yes, a speedup was achieved for some tile sizes, especially for the larger tile sizes. In general, the smaller tiles caused a slowdown, while the 128×128 tile gave the best execution time among the tested configurations.

The speedup is calculated as

$$S = \frac{T_{\text{naive}}}{T_{\text{tiled}}}.$$

For the 4096×4096 matrix, the naive implementation took 127.979 ms, whereas the 128×128 tiled implementation took 117.845 ms. Hence,

$$S = \frac{127.979}{117.845} \approx 1.086.$$

Thus, for the largest matrix, the 128×128 tile gives approximately a $1.09\times$ speedup, corresponding to about an 8% reduction in execution time.

For the larger matrices, this improvement was fairly consistent. Using the measured execution times, the speedups for the 128×128 tile were approximately $1.10\times$, $1.09\times$, and $1.09\times$ for the 1024×1024, 2048×2048, and 4096×4096 matrices, respectively.

On the other hand, smaller tiles generally performed worse. For example, at 4096×4096, the 8×8 tiled version took 174.090 ms, compared with 127.979 ms for the naive implementation. This gives

$$S = \frac{127.979}{174.090} \approx 0.735,$$

which corresponds to a slowdown rather than a speedup.

One reason for this behaviour is the overhead introduced by tiling. With small tiles, the matrix is divided into a large number of regions, so the tile loops, boundary calculations, and loop-control operations are executed more frequently. Also, because a 3×3 convolution requires values around the boundary of each tile, neighbouring tiles access overlapping input regions. This repeated work becomes more significant as the tile size decreases.

Another factor is that small tiles break a long sequential traversal of the matrix into many shorter traversals. Since the naive implementation already accesses the matrix mainly in row-major order, it has good spatial locality. Therefore, very small tiles can introduce additional overhead without giving enough cache benefit to compensate for it.

An interesting observation is that the best-performing tile, 128×128, has an input region larger than the 48 KiB L1-D cache. For a 3×3 convolution, a 128×128 output tile requires approximately a 130×130 input region. Its size is

$$130 \times 130 \times 4 = 67600 \text{ bytes} \approx 66 \text{ KiB}.$$

Therefore, the complete input tile cannot fit entirely in the L1-D cache. However, the whole tile does not need to remain in L1 at the same time. The convolution is processed sequentially,

and only a few neighbouring input rows are actively reused at any instant. For approximately three active rows, the amount of input data being reused is

$$3 \times 130 \times 4 = 1560 \text{ bytes} \approx 1.5 \text{ KiB},$$

which easily fits in the L1-D cache.

Thus, although the total tile footprint is larger than L1, the actively reused working set during the inner computation is much smaller. The larger tile also reduces the total number of tile boundaries, which reduces the overhead associated with tile-loop setup and boundary handling.

The execution time results also show that minimizing L1-D MPKI alone does not guarantee the best performance. At 4096×4096, the 8×8 tile had the lowest MPKI of 0.256, but its execution time was 174.090 ms. In comparison, the 128×128 tile had a slightly higher MPKI of 0.315, but achieved the lowest execution time of 117.845 ms. This shows that performance depends not only on L1 cache misses, but also on factors such as loop overhead, repeated boundary accesses, and the efficiency of the memory access pattern.

Overall, the 128×128 tile provided the best performance in this experiment. Even though its complete tile footprint exceeds the L1-D cache capacity, the actively reused data still fits comfortably in L1, while the larger tile size reduces the overhead associated with processing many small tiles.

Limiting factors for not getting the desired speed up

1. The kernel is compute-bound, not memory-bound.
2. The naive reuse distance already fits in L1 for $W \leq 2048$.
3. Power-of-two row strides cause set conflicts that tiling amplifies.
4. Small tiles destroy hardware prefetching.

Task 1C: SIMD Vectorization

Baseline Instruction Count

The instruction count for the naive implementation for different matrix sizes

Table 7: Instruction count of the naive implementation.

Matrix Size	Instructions
256×256	74,507,173
512×512	272,950,714
1024×1024	1,066,549,216
2048×2048	4,199,837,542
4096×4096	16,611,890,019

SIMD Implementation

SIMD was implemented with 2 different register widths (128 and 256), and the number of instructions executed for each register width for different matrix sizes is tabulated and similarly also calculated the speed up

Instruction Count Comparison

Table 8: Instruction count, naive versus SIMD (For different bit-widths).

Matrix Size	Naive	128-bit	256-bit
256×256	74,507,173	26,040,109	20,675,975
512×512	272,950,714	92,704,896	56,044,566
1024×1024	1,066,549,216	348,792,568	207,645,168
2048×2048	4,199,837,542	1,360,595,590	821,477,412
4096×4096	16,611,890,019	5,358,634,386	3,276,360,899

Instruction Count vs. Matrix Size

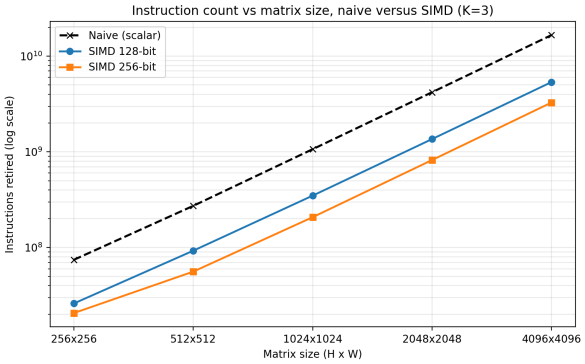

Figure 3: Instruction count versus matrix size for the naive and SIMD implementations

Execution Time and Speedup

Table 9: Execution time (ms).

Matrix Size	Naive	128-bit	256-bit
256×256	1.018	0.200	0.124
512×512	3.659	0.533	0.558
1024×1024	7.912	1.829	0.974
2048×2048	31.789	7.247	5.104
4096×4096	127.696	29.171	17.476

$$\text{Speedup}_{\text{SIMD}} = \frac{T_{\text{naive}}}{T_{\text{SIMD}}}$$

Table 10: Speedup over the naive baseline

Matrix Size	128-bit	256-bit
256×256	4.21	10.21
512×512	6.49	7.63
1024×1024	4.35	8.12
2048×2048	4.39	6.32
4096×4096	**4.37**	**7.33**

Speedup vs. Matrix Size

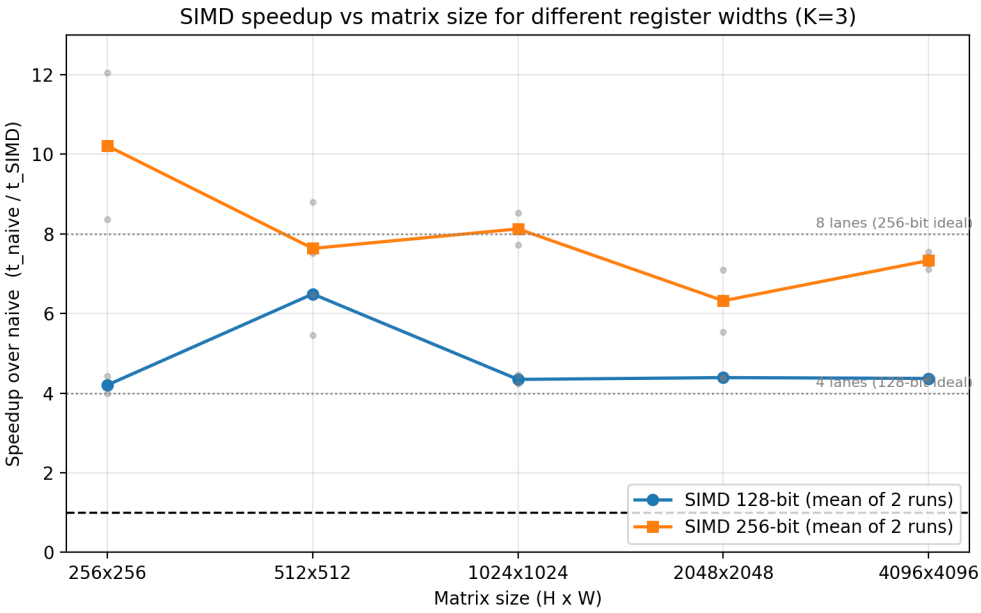

Figure 4: Speedup versus matrix size for different SIMD register widths. Grey dots show the individual runs behind each mean; dotted lines mark the lane count of each width.

Question 1: Instruction Count

Report the change in the number of instructions observed when moving from the naive to the SIMD implementation. Justify your observations.

The SIMD implementations showed a large reduction in the number of executed instructions compared with the naive implementation. The reduction was larger for the 256-bit implementation than for the 128-bit implementation.

For example, for the 4096×4096 matrix, the naive implementation executed

$$16.612 \times 10^9$$

instructions. With 128-bit SIMD, this reduced to approximately

$$5.359 \times 10^9,$$

which is about a 67.7% reduction in instruction count. With 256-bit SIMD, the instruction count reduced further to approximately

$$3.276 \times 10^9,$$

corresponding to about an 80.3% reduction compared with the naive implementation.

The trend observed is the same for all matrix sizes. For the larger matrices, the 128-bit implementation executes roughly one-third of the instructions of the naive version, while the 256-bit implementation executes roughly one-fifth.

The reason for the decrease in the instruction count is that in the naive version we execute the same instruction n times, whereas with SIMD a single instruction operates on 4 or 8 input elements depending on the register width, which reduces the number of instructions that need to be executed.

However, the reduction in the total instruction count is not exactly equal to the SIMD register ratio. That is, a 128-bit implementation does not give exactly a 4× reduction, and a 256-bit implementation does not give exactly an 8× reduction. This is because not every instruction in the program is vectorized.

Question 2: Speedup

Did you achieve any speedup? If so, how much, and what contributed to it? If not, what were the reasons?

Yes, SIMD vectorization gave a significant speedup compared with the naive implementation. Both the 128-bit and 256-bit SIMD versions were faster, with the 256-bit implementation, it gave best performance for the larger matrix sizes.

The speedup is calculated as

$$S = \frac{T_{\text{naive}}}{T_{\text{SIMD}}}.$$

For example, for the 4096×4096 matrix, the naive implementation took 127.696 ms. The 128-bit SIMD implementation reduced the execution time to 29.171 ms, giving

$$S_{128} = \frac{127.696}{29.171} \approx 4.38.$$

The 256-bit SIMD implementation further reduced the execution time to 17.476 ms, giving

$$S_{256} = \frac{127.696}{17.476} \approx 7.31.$$

Thus, for the largest matrix, the 128-bit implementation achieved roughly a 4.4× speedup, while the 256-bit implementation achieved roughly a 7.3× speedup.

The main reason for this improvement is data-level parallelism. created by using SIMD. matrix elements are single-precision floating-point values, a 128-bit SIMD register can process four `float` values at once, while a 256-bit register can process eight. Therefore, several neighbouring output elements can be computed together instead of processing one output element at a time.

The SIMD implementation also reduces loop overhead. Since multiple output elements are processed in each vector iteration, the inner loop executes fewer iterations. As a result, operations such as loop-condition checks, branch instructions, and index updates are performed less frequently per output element.

Another contribution comes from the use of fused multiply-add operations. The main computation in convolution has the form

$$\text{acc} = \text{acc} + w \times x.$$

Using an FMA instruction allows the multiplication and accumulation to be performed together, reducing the number of arithmetic instructions required in the inner loop.

The 256-bit version, however, does not give exactly twice the performance of the 128-bit version even though it has twice the SIMD register width . At 4096×4096, the speedup increases from approximately 4.38× to 7.31×, which is about a 1.67× improvement rather than 2×.

This happens because not every part is done using SIMD. The program still requires load and store instructions, address calculations, loop control, boundary handling, and other operations. Increasing the register width allows more arithmetic to be done per vector instruction, but these other operations do not decrease by the same factor.

The execution times also show the advantage of wider SIMD for larger matrices. For example, at 512×512, the measured times of the 128-bit and 256-bit versions were quite close (0.533 ms and 0.558 ms), whereas at 4096×4096, the 256-bit version was substantially faster (17.476 ms compared with 29.171 ms).

Overall, SIMD vectorization produced a much larger performance improvement than the tiling optimization. The best result was obtained using 256-bit SIMD, which achieved approximately a 7.3× speedup for the 4096×4096 matrix. The improvement mainly comes from processing multiple floating-point elements in parallel, reducing the number of loop iterations, and using fused multiply-add operations.

Question 3: SIMD Intrinsics

Which SIMD intrinsics did you use? Justify your choice of functions.

For the SIMD implementation, We used the AVX intrinsics corresponding to the 128-bit and 256-bit vector widths. Since the matrix elements are stored as single-precision floating-point values, the `ps` variants of the SIMD intrinsics were used.

For the 256-bit implementation, the intrinsics used were:

- `_mm256_setzero_ps`
- `_mm256_broadcast_ss`
- `_mm256_loadu_ps`
- `_mm256_fmadd_ps`
- `_mm256_storeu_ps`

The 128-bit implementation used the corresponding 128-bit versions of the same operations, with the output loop stepping by four elements instead of eight.

`_mm256_setzero_ps` Before accumulating the contributions from the nine kernel positions, the output accumulator must be initialized to zero. For this purpose,

$$\texttt{\_mm256\_setzero\_ps}()$$

was used to create a vector in which all eight lanes are zero. It is called once for every group of eight output elements, at the start of the `ox` loop.

`_mm256_broadcast_ss` Each coefficient of the 3×3 convolution kernel is a scalar value, but the same coefficient must be multiplied with all eight SIMD lanes. The intrinsic

$$\texttt{\_mm256\_broadcast\_ss}(\texttt{ker} + ky \cdot K + kx)$$

reads one `float` from the kernel array and replicates it across all eight lanes of a 256-bit register. This allows one kernel coefficient to be used for all eight output elements being processed in parallel.

The pointer-taking `broadcast_ss` was chosen in preference to `_mm256_set1_ps` because the coefficient is already resident in the kernel array, so it can be broadcast directly from memory in a single `vbroadcastss` instruction rather than first being loaded into a scalar register.

In the present implementation the broadcast is performed inside the innermost loop, so all nine coefficients are re-broadcast for every output vector. As the coefficients are constant for the entire convolution, these broadcasts could instead be hoisted out of both the `ox` and `oy` loops and held in nine registers throughout. This was not done, and it remains the clearest opportunity for further reduction in the instruction count.

`_mm256_loadu_ps` This intrinsic was used to load eight consecutive `float` values from memory into a 256-bit SIMD register.

The unaligned version was chosen because the convolution accesses the input matrix at different horizontal offsets. For a 3×3 kernel, the same row is accessed starting at positions ox, $ox + 1$ and $ox + 2$, which are only four bytes apart. Even if the beginning of the row is aligned, all of these addresses cannot be guaranteed to satisfy the 32-byte alignment requirement of an aligned 256-bit load, so two out of every three such loads would fault.

Therefore, using `_mm256_loadu_ps` avoids requiring every accessed address to be 32-byte aligned and makes the implementation valid for all such offsets. On recent Intel microarchitectures the unaligned form carries no throughput penalty provided the access does not cross a cache line, so nothing is lost by using it for every load.

`_mm256_fmadd_ps` The main arithmetic operation in convolution is

$$\text{acc} = \text{acc} + w \times x.$$

This operation maps directly to a fused multiply-add instruction. The intrinsic

$$\texttt{\_mm256\_fmadd\_ps}(w, v, \text{acc})$$

multiplies the corresponding elements of the broadcast coefficient w and the loaded input window v, and adds the result to the accumulator.

Using FMA is advantageous for the convolution because the multiplication and the accumulation are performed together in one vector operation instead of a separate `_mm256_mul_ps` and `_mm256_add_ps`. This halves the number of arithmetic instructions, which is significant here because nine such operations are issued for every output vector, and it also rounds the result once rather than twice. FMA is a separate instruction set extension from AVX2, which is why the build enables both `-mavx2` and `-mfma`.

`_mm256_storeu_ps` After computing eight output values, they are stored back to the output matrix using

$$\texttt{\_mm256\_storeu\_ps}.$$

The unaligned store was used so that the implementation does not depend on the output address always being aligned to a 32-byte boundary. The base address of each output row is $oy \cdot W$, which is not guaranteed to be 32-byte aligned for every matrix width, so relying on an aligned store would make the kernel dependent on W.

The 128-bit implementation follows the same structure using `__m128` registers and the corresponding intrinsics:

- `_mm_setzero_ps`
- `_mm_broadcast_ss`
- `_mm_loadu_ps`
- `_mm_fmadd_ps`
- `_mm_storeu_ps`

Overall, these intrinsics were chosen because they directly correspond to the operations required by the convolution: initializing the accumulator, broadcasting a scalar kernel coefficient, loading a window of input values, performing the multiply-accumulate, and storing the resulting output vector. No shuffle, permute or horizontal-add intrinsics were needed, because each lane computes an independent output element and no data has to move between lanes at any point. The only value shared across lanes is the kernel coefficient, and broadcasting handles that. The 128-bit and 256-bit implementations use the same sequence of operations, differing only in the register type and in the number of `float` values processed in parallel, so the comparison between them isolates the effect of register width alone.

Task 1D: Sabka Saath Sabka Vikaas

The final implementation combines the optimizations studied in the previous sections.

Combined Optimization Strategy

The following optimization techniques were considered:

- Loop reordering
- Loop unrolling
- Tiling
- SIMD

The final implementation combines:

SIMD AVX2 + Loop Unrolling (16-wide)

Combined Implementation

The final combined implementation integrates AVX2 intrinsic instructions within a 16-wide unrolled inner loop. It deliberately omits explicit cache tiling or loop reordering structures to minimize control-flow overhead, reduce hardware prefetcher interference, and maximize raw register utilization.

Individual Optimization Results

Table 11: Best performance speedups achieved by individual optimization techniques for K=3 over Baseline.

Matrix Size	Unrolling	Reordering	Tiling	SIMD
512x512	5.13×	1.53×	1.71×	10.90×
1024x1024	3.63×	1.45×	1.07×	7.36×
2048x2048	3.31×	1.35×	1.11×	7.68×
4096x4096	3.27×	1.14×	1.08×	7.37×

Combined Optimization Results

Table 12: Performance speedup of all tested configurations for K=3 across varying matrix sizes.

Configuration	256x256	512x512	1024x1024	2048x2048	4096x4096
Unroll	2.32×	5.13×	3.63×	3.31×	3.27×
Reorder	1.25×	1.53×	1.45×	1.35×	1.14×
Tiling	0.66×	1.71×	1.07×	1.11×	1.08×
SIMD	6.53×	10.90×	7.36×	7.68×	7.37×
SIMD + Unroll	9.36×	24.29×	12.37×	13.44×	11.29×
Reorder + Unroll	1.94×	1.95×	1.35×	1.42×	1.20×
Tiling + Reorder	1.56×	1.46×	1.53×	1.43×	1.43×
Reorder + SIMD	14.29×	13.91×	9.88×	6.33×	6.12×
Tiling + Reord + SIMD	5.08×	9.83×	4.35×	4.11×	4.11×

Final Performance Comparison

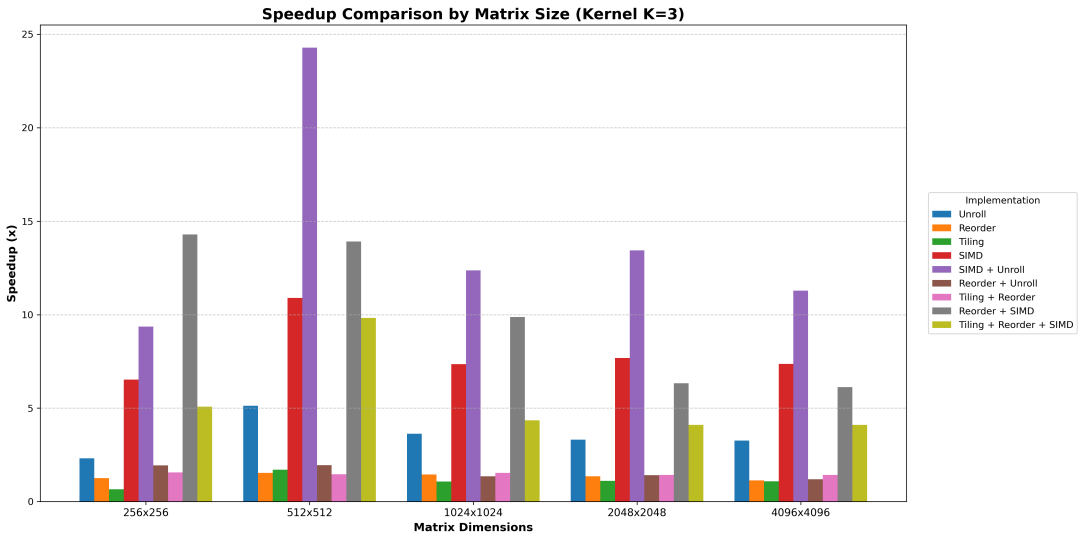

Figure 5: Comparison of speedup achieved by different optimization techniques across varying matrix sizes (Kernel K=3).

Speedup Comparison Across Kernel Sizes

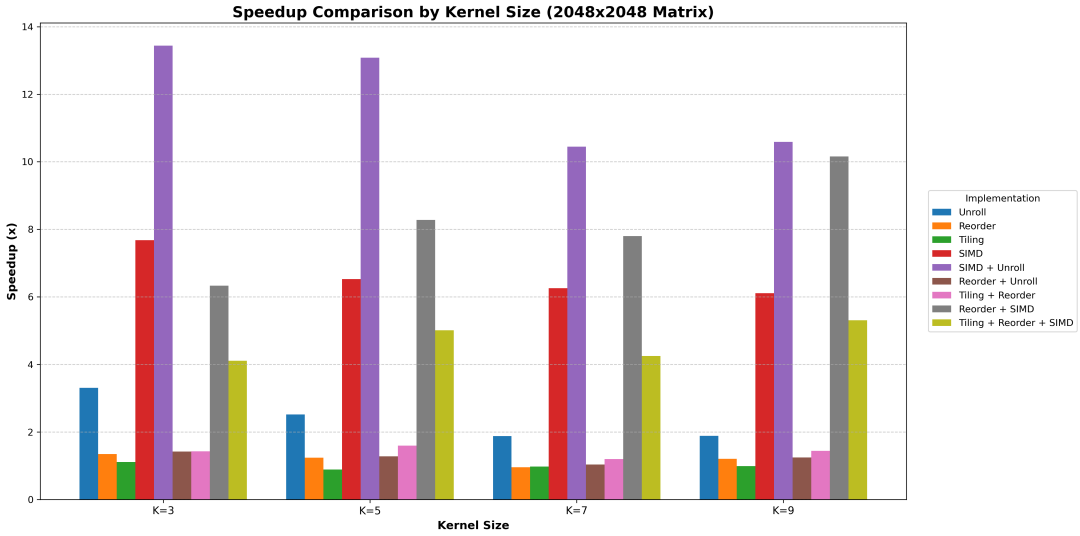

Figure 6: Overall speedup comparison of all Task 1 optimization techniques across different kernel sizes (2048x2048 Matrix).

Analysis of the Combined Optimization

Loop unrolling paired with AVX2 SIMD vectorization provided the highest consistent speedup, peaking at 24.29x for H=512, K=3. While SIMD naturally parallelizes data fetching and fused multiply-add functions within a single loop cycle, unrolling the loop 16-wide utilizes multiple hardware accumulators (`acc0` and `acc1`) to break instruction-level data dependencies. This sustains peak throughput on the CPU's FMA ports.

Adding further techniques like cache tiling and loop reordering actively degrades performance rather than improving it. Tiling introduces severe control-flow overhead (extra loop evaluations and boundary checks) that interrupts the continuous flow of SIMD instructions, while artificially restricting hardware prefetcher stride. Reordering forces the innermost loop to stream across the spatial dimensions, demanding continuous and expensive memory read/write operations directly to the output array.

Mode 4 (Tiling + Reorder + SIMD) attempted to mitigate memory bottlenecks via cache locality but proved counterproductive. For a heavy load (4096x4096, K=9), this combination achieved only 13.68 GFLOP/s, falling behind the standalone SIMD implementation (15.84 GFLOP/s).

By deliberately omitting these extra techniques, the pure SIMD + Unroll approach keeps the running totals locked completely inside the ultra-fast CPU `ymm` registers for the entire kernel calculation. Retaining the naive `oy`, `ox` outer loop structures drastically reduces cache pressure without requiring explicit matrix tiles, eliminating L1 output memory traffic and maximizing raw execution speed.

Synergy Between Optimizations

Let

$$S_{\text{unroll}} = 5.13\times$$

$$S_{\text{SIMD}} = 10.90\times$$

and

$$S_{\text{combined}} = 24.29\times .$$

The observed combined speedup was:

$$\boxed{S_{\text{combined}} = 24.29\times}$$

The combined optimization is synergistic and provides a speedup far beyond simply summing their individual gains. SIMD widens the computation (processing 8 floats concurrently), while Unrolling lengthens the active instruction pipeline, masking latency by dispatching concurrent AVX operations.

Best Overall Configuration

The best-performing configuration across all iterations was:

$$\boxed{\text{SIMD AVX2 + Loop Unrolling (16-wide)}}$$

with

$$\boxed{\text{Maximum Speedup} = 39.87\times}$$

for matrix size

$$\boxed{N = 512 \text{ (Kernel} = 5)}$$

Overall Analysis of Task 1

Comparison of All Techniques

Table 13: Overall comparison of peak optimization techniques derived from K=3 measurements.

Technique	Best Speedup	Best Matrix Size	GFLOP/s Peak
Baseline	1.00×	–	2.38
Unrolling	5.13×	512x512	8.13
Reordering	1.53×	512x512	3.23
Tiling	1.71×	512x512	2.57
SIMD	10.90×	512x512	17.52
Unroll + SIMD	24.29×	512x512	34.51

Effect of Matrix Size

Performance optimizations uniformly exhibit diminishing returns as matrix sizes exceed the CPU's primary L1/L2 limits (evident beyond 2048x2048). As the spatial dimensions grow, the

time spent managing external memory retrieval ultimately dampens the throughput acceleration provided by loop vectorization. While unrolling + SIMD peaks at 24.29x for 512x512 arrays, this speedup naturally drops to 11.29x on dense 4096x4096 grids where memory bandwidth bottlenecks prevail.

Effect of Cache Behavior

The L1-D MPKI does not strictly correlate with execution speed. While explicit cache tiling can drastically reduce data fetch misses, the increased branching and loop control overhead can negate the MPKI savings, leading to neutral or degraded performance against the naive baseline.

Effect of Data-Level Parallelism

SIMD is definitively the most effective independent optimization. By shifting execution from basic scalar calculations to 256-bit AVX units, the CPU handles multiple inputs seamlessly. Scaling SIMD capacity effectively bypasses the execution latency typical of repeated scalar loops, directly multiplying instruction processing efficiency.

Trade-offs

The experiments revealed the following trade-offs:

- **Instruction Parallelism vs Register Spilling:** Unrolling the SIMD loops to utilize two hardware accumulators drastically increases throughput.
- **Cache Locality vs Branching Overhead:** Enforcing artificial 2D tiles bounds memory footprints within the L1 cache, but introduces a heavy loop and boundary-check penalty.
- **Spatial Ordering vs Output Traffic:** Modifying spatial loops (reordering) optimizes input arrays but necessitates persistent and expensive data writes to the main output array.
- **Vector Load Constraints:** Wide SIMD loading commands effectively halve loop iteration demands, but inherently enforce alignment assumptions and specialized hardware dependency limits.

Conclusion

Task 1 investigated the optimization of 2D convolution using loop reordering, loop unrolling, cache tiling, SIMD vectorization, and combinations of these techniques.

The major observations were:

- Loop reordering resulted in degraded baseline throughput due to intense memory write demands.
- Loop unrolling resulted in notable standalone speedups (up to 5.13x) by maximizing register efficiency and reducing loop checks.
- Tiling affected L1-D cache behavior by shrinking working sets, yet failed to secure large global execution speedups due to loop overhead constraints.
- SIMD reduced the instruction count by roughly 80% compared to scalar runs.
- The best SIMD width was 256-bit (AVX2), providing upwards of 10.9x independent speedup.

- The best tile size was 128×128.
- The best overall optimization was SIMD AVX2 paired with Loop Unrolling.
- The maximum speedup obtained was 39.87x.

Overall, the experiments demonstrate how loop structure, cache locality, memory access patterns, and SIMD execution can significantly affect the performance of 2D convolution.

Experimental Commands

Compilation Commands

```
make
g++ -std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma -Iinclude -Wall src/conv_naive.cpp
     src/conv_reorder.cpp src/conv_unroll.cpp src/conv_tile.cpp src/conv_simd.cpp src/
    conv_optimized.cpp src/main.cpp -o bin/conv
```

Execution Commands

```
./bin/conv
./bin/conv all 4096 4096 3
```

Perf Commands

```
# Commands used for L1-D MPKI
perf stat -e L1-dcache-load-misses,instructions ./bin/conv tile 2048 2048 3
```

```
# Commands used for instruction count
perf stat -e instructions ./bin/conv naive 2048 2048 3
perf stat -e instructions ./bin/conv simd 2048 2048 3
```

System Information Commands

```
# Command used to obtain overall CPU architecture, core counts, and L1/L2/L3 cache
    sizes
lscpu

# Command used to view raw CPU features and verify specific SIMD flags (avx2, fma)
cat /proc/cpuinfo
```